A Equal Split Prefixes

Time Limit: 3 second(s) | Memory Limit: 1024 MB

You are given an array `a` of length `n`. A split position `i` is valid if `1 <= i < n` and the sum of the first `i` elements equals the sum of the remaining elements.

Compute the number of valid split positions.

Input

- The first line contains one integer `n`.
- The second line contains `n` integers `a1, a2, ..., an`.

Output

Print the number of valid split positions.

Constraints

- `2 <= n <= 200000`
- `-10^9 <= ai <= 10^9`

Examples

Sample Input	Sample Output
6 1 2 3 3 2 1	1

Example Explanation

The array is `[1, 2, 3, 3, 2, 1]` with total sum `12`. The only valid split position is `i = 3`: the prefix `[1, 2, 3]` sums to `6` and the suffix `[3, 2, 1]` also sums to `6`. No other position yields equal halves, so the answer is `1`.

B Cyclic Match

Time Limit: 3 second(s) | Memory Limit: 1024 MB

You are given two strings `s` and `t` of equal length. In one operation, you may perform a left cyclic rotation on `s`, moving its first character to the end.

Find the minimum number of rotations needed to transform `s` into `t`, or report `-1` if it is impossible.

Input

- The first line contains string `s`.
- The second line contains string `t`.

Output

Print the minimum number of left rotations, or `-1`.

Constraints

- `1 <= |s| = |t| <= 200000`
- Both strings consist of lowercase English letters.

Examples

Sample Input	Sample Output
abac acab	2

Example Explanation

With `s = "abac"` and `t = "acab"`: after **1** left rotation `s` becomes `"baca"`; after **2** rotations it becomes `"acab"`, which equals `t`. So the answer is `2`.

C Portal Grid

Time Limit: 3 second(s) | Memory Limit: 1024 MB

You are given a grid containing walls, empty cells, a start cell `S`, an exit cell `E`, and portal cells marked by lowercase letters.

You may move up, down, left, or right to a non-wall cell with cost `1`. If you stand on a portal cell with letter `c`, you may jump to any other cell with the same letter, also with cost `1`. However, the portal network for each letter may be activated at most once in the entire search.

Find the minimum cost to reach `E` from `S`, or print `-1` if impossible.

Input

- The first line contains integers `n` and `m`.
- The next `n` lines describe the grid.

Output

Print the minimum cost.

Constraints

- `1 <= n, m <= 2000`
- The total number of cells over all tests is intended to fit an `O(nm)` solution.

Examples

Sample Input

```
4 5
S.a..
##.#.
a..#E
.....
```

Sample Output

```
6
```

Example Explanation

The grid is:

```
S . a . .
# # . # .
```

```
a . . # E
. . . . .
```

`S` is at `(0,0)` and `E` is at `(2,4)`. The shortest path avoids portals entirely: move right along row 0 from `(0,0)` to `(0,4)` (4 steps), then down to `(1,4)` (1 step), then down to `(2,4)=E` (1 step) — total cost **6**. Portal cells can be walked through without activating them.

D Critical Route In A DAG

Time Limit: 3 second(s) | Memory Limit: 1024 MB

You are given a directed acyclic graph with weighted edges. Find the maximum possible total weight of a path from node `1` to node `n`, and the number of distinct maximum-weight paths.

If node `n` is unreachable from node `1`, print `-1 0`.

Input

- The first line contains integers `n` and `m`.
- Each of the next `m` lines contains `u v w`, describing a directed edge from `u` to `v` with weight `w`.
- The graph is guaranteed to be acyclic.

Output

Print two integers: the maximum total weight and the number of maximum-weight paths modulo `1000000007`.

Constraints

- `2 <= n <= 200000`
- `1 <= m <= 300000`
- `1 <= u, v <= n`
- `|w| <= 10^9`

Examples

Sample Input

```
5 6
1 2 3
1 3 2
2 4 4
3 4 5
4 5 1
2 5 4
```

Sample Output

```
8 2
```

Example Explanation

The two heaviest paths from node `1` to node `5` are: - `1 → 2 → 4 → 5` with total weight `3 + 4 + 1 = 8` - `1 → 3 → 4 → 5` with total weight `2 + 5 + 1 = 8`

Both achieve the maximum weight of `8`, and the path `1 → 2 → 5` (weight `7`) is suboptimal. The answer is `8 2`.

E Weighted Job Selection

Time Limit: 3 second(s) | Memory Limit: 1024 MB

You are given `n` jobs. Job `i` starts at time `li`, ends at time `ri`, and yields profit `pi`. You may choose a subset of non-overlapping jobs. A job ending at time `x` does not overlap a job starting at time `x`.

Find the maximum total profit.

Input

- The first line contains integer `n`.
- Each of the next `n` lines contains `li ri pi`.

Output

Print the maximum total profit.

Constraints

- `1 <= n <= 200000`
- `0 <= li <= ri <= 10^9`
- `1 <= pi <= 10^9`

Examples

Sample Input

```
6
1 3 5
2 5 6
4 6 5
6 7 4
5 8 11
7 9 2
```

Sample Output

```
17
```

Example Explanation

The 6 jobs are `(l, r, profit)`: `(1,3,5)`, `(2,5,6)`, `(4,6,5)`, `(6,7,4)`, `(5,8,11)`, `(7,9,2)`. The optimal selection is job 2 `(2,5,6)` and job 5 `(5,8,11)`: they do not overlap since job 2 ends at `5` and job 5 starts at `5` (ending and starting at the same time is allowed). Total profit = `6 + 11 = 17`.

Problem E: Weighted Job Selection

F Time-Locked Bridges

Time Limit: 3 second(s) | Memory Limit: 1024 MB

There are `n` cities and no roads initially. Then `q` events happen. Each event is one of:

- `+ u v` : add an undirected road between `u` and `v`
- `- u v` : remove the currently active road between `u` and `v`
- `?` : ask for the current number of connected components

Every removed road is guaranteed to be currently active, and no road is added twice without being removed first.

Print the answer for every `?` event.

Input

- The first line contains integers `n` and `q`.
- The next `q` lines describe the events.

Output

For each query event, print the number of connected components.

Constraints

- `1 <= n <= 200000`
- `1 <= q <= 200000`

Examples

Sample Input

```
5 8
?
+ 1 2
+ 2 3
?
- 1 2
?
+ 4 5
?
```

Sample Output

```
5
3
4
3
```

Example Explanation

Trace of events with `n = 5` cities (initially all isolated, 5 components): 1. `?` → **5** components (no roads yet) 2. `+ 1 2` → connect 1 and 2 → 4 components 3. `+ 2 3` → connect 2 and 3 (merges with {1,2}) → 3 components 4. `?` → **3** components 5. `- 1 2` → remove road 1–2; cities 1, 2, 3 are still connected via 2–3, so only {1} separates… wait, there is no 1–3 edge. Removing 1–2 disconnects 1 from {2,3}: 4 components. 6. `?` → **4** components 7. `+ 4 5` → connect 4 and 5 → 3 components 8. `?` → **3** components

Expedition Resupply

Time Limit: 3 second(s) | Memory Limit: 1024 MB

An expedition team treks across a polar ice sheet. The route consists of n checkpoints. At checkpoint i the team either gains s_i units of fuel (if s_i > 0) or consumes |s_i| units (if s_i < 0). The team starts with 0 fuel and fuel must never drop below 0 at any point. They may choose any single checkpoint as the starting point and travel in circular order (wrapping from n back to 1). Find the smallest valid starting index (1-indexed). Print -1 if no valid start exists.

Input

The first line contains integer n.

The second line contains n integers s_1, s_2, ..., s_n.

Output

Print the smallest valid starting index (1-indexed), or -1 if impossible.

Constraints

- $1 \le n \le 200000$
- $|s_i| \le 10^9$

Examples

Sample Input

```
5
-2 3 -1 4 -3
```

Sample Output

```
2
```

Example Explanation

Values: [-2, 3, -1, 4, -3]. Total sum = 1 ≥ 0, so a valid starting index must exist. Starting at index 1: fuel immediately becomes -2, which is invalid. Starting at index 2: fuel progresses as 3 → 2 → 6 → 3 → 1 (after wrapping to index 1). Fuel never drops below 0. Index 2 is the smallest valid start.

Fiber Tangle

Time Limit: 3 second(s) | Memory Limit: 1024 MB

A telecommunications engineer is laying n fiber optic cables underground. Each cable is a straight segment defined by its two endpoints. Two cables conflict if they cross at a point that is strictly interior to both cables. Touching at a shared endpoint is not a conflict. Count the number of conflicting cable pairs. The answer may be large — print it modulo 10^{9+7}.

Input

The first line contains integer n.

Each of the next n lines contains four integers x_1 y_1 x_2 y_2 — the endpoints of cable i.

Output

Print the count of conflicting pairs modulo 10^{9+7}.

Constraints

- $1 \le n \le 3000$
- $|x_i|, |y_i| \le 10^9$
- No three cables pass through the same single point.
- No cable degenerates to a point.

Examples

Sample Input

```
4
0 0 6 4
0 4 6 0
2 0 4 5
7 0 7 6
```

Sample Output

```
3
```

Example Explanation

Four cables: C1 from (0,0) to (6,4), C2 from (0,4) to (6,0), C3 from (2,0) to (4,5), C4 from (7,0) to (7,6).

C1 and C2 cross at (3, 2) — conflict. C1 and C3 cross at approximately (30/11, 20/11) — conflict. C2 and C3 cross at approximately (54/19, 40/19) — conflict. C4 lies entirely at x = 7 and does not intersect any other cable. All three intersection points are distinct, satisfying the no-three-concurrent constraint. Total: 3.

Synchronized Clocks

Time Limit: 3 second(s) | Memory Limit: 1024 MB

A satellite constellation consists of n satellites. Satellite i transmits a beacon pulse every p_i seconds, with its first pulse at time o_i (so it pulses at times o_i, o_i + p_i, o_i + 2p_i, ...). Ground control needs to find the earliest time t ≥ 0 at which all n satellites pulse simultaneously. If no such moment exists, print NO.

The answer may be very large. If a solution exists, print YES followed by the answer modulo 10^{9+7} on the same line.

Input

The first line contains integer n.

Each of the next n lines contains two integers p_i and o_i.

Output

Print YES t (where t is the answer modulo 10^{9+7}) if a solution exists, or NO otherwise.

Constraints

- $1 \le n \le 100$
- $1 \le p_i \le 10^9$
- $0 \le o_i < p_i$

Examples

Sample Input

```
3
3 1
4 2
7 3
```

Sample Output

```
YES 10
```

Example Explanation

Satellite 1 pulses at 1, 4, 7, 10, 13, ... (period 3, offset 1). Satellite 2 pulses at 2, 6, 10, 14, ... (period 4, offset 2). Satellite 3 pulses at 3, 10, 17, ... (period 7, offset 3). At t = 10: 10 mod 3 = 1 ✓, 10 mod 4 = 2 ✓, 10 mod 7 = 3 ✓. All three satellites pulse simultaneously. The answer is 10.

Spectrum Windows

Time Limit: 3 second(s) | Memory Limit: 1024 MB

A radio telescope records n frequency readings. Reading i has value a_i. Analysts submit q operations of three types:

- 1 i x : update reading i to frequency x
- 2 l r : count the number of distinct frequency values in readings l to r (inclusive)
- 3 l r f : count how many readings in [l, r] have frequency exactly f

Operations are interleaved arbitrarily. Each query reflects the array state after all preceding updates.

Input

The first line contains integers n and q.

The second line contains n integers a_1, ..., a_n.

Each of the next q lines contains one operation.

Output

For each type 2 and type 3 operation, print the answer on a separate line.

Constraints

- $1 \le n \le 100000$
- $1 \le q \le 100000$
- $1 \le a_i, x, f \le 10^9$
- $1 \le l \le r \le n$
- O(n sqrt(n)) solutions will receive partial credit only.

Examples

Sample Input

```
6 7
5 3 5 7 3 5
2 1 6
3 1 6 5
1 4 5
2 1 6
3 1 3 5
1 6 7
2 3 6
```

Sample Output

```
3
3
2
2
3
```

Example Explanation

Initial array: [5, 3, 5, 7, 3, 5].

Op 1 — Q2(1,6): distinct values in [5,3,5,7,3,5] = {3,5,7} → 3.

Op 2 — Q3(1,6,5): count of 5 in full array = positions 1,3,6 → 3.

Op 3 — Update a[4] = 5: array becomes [5, 3, 5, 5, 3, 5].

Op 4 — Q2(1,6): distinct values in [5,3,5,5,3,5] = {3,5} → 2.

Op 5 — Q3(1,3,5): count of 5 in [5,3,5] = positions 1,3 → 2.

Op 6 — Update a[6] = 7: array becomes [5, 3, 5, 5, 3, 7].

Op 7 — Q2(3,6): distinct values in [5,5,3,7] = {3,5,7} → 3.

Full-credit solutions should support interleaved point updates and both query types in $O((n+q)\log^2 n)$ time. One valid full-credit approach is an offline CDQ divide-and-conquer method that handles updates and range queries together. Simpler $O(\sqrt{n})$ per query methods receive partial credit only.